

Atomic Motion Coordinate for Language-Steerable and Force-Responsive Manipulation

Anonymous

Abstract—Can changing only the language instruction redirect a vision-language-action (VLA) policy’s end effector, or does the visually driven motion prior dominate? We present Atomic Motion Coordinate (AMC), a geometry-grounded coordinate for steerable and force-responsive manipulation. Each arm owns thirteen signed translation, rotation, and hold atoms grounded from text and forward kinematics with vision withheld, and the coordinate is injected into every action-expert block via a cross-attention residual. Contact history modulates the same coordinate through a bounded spherical residual that is recomputed from a fixed nominal latent to regenerate only the unexecuted horizon suffix. Across 7,520 per-policy offline horizon interventions, opposite-atom separation reaches 92.5/83.1% (single/dual) versus 39.1/24.0% for LA4VLA-style. Across 50 real-robot trials per task, AMC raises out-of-distribution (OOD) fruit progress from 60.5% to 87.8%; force adaptation raises Plug/Vase from 59.0/71.6% to 78.5/75.2%.

I. INTRODUCTION

Generalist vision–language–action (VLA) policies such as OpenVLA, π_0 , and $\pi_{0.5}$ turn instructions and camera streams into continuous control across a widening range of manipulation tasks [1], [2], [3]. Demonstrations couple what is said, seen, and done, so high task success can still reflect a weak language prior over a visually driven routine. Recent work finds that task targets are learned more readily from visual specifications than from text, and proposes sparse point flows as a more learnable instruction channel [4]. Rather than replacing language, we ask what it controls. **With the same observation and robot state, does changing only the instruction move the predicted end effector in the requested direction rather than continue the demonstrated motion?**

We therefore freeze observation, robot state, and inference settings while varying only the prompt. A fine-tuned $\pi_{0.5}$ does respond—pretraining supplies common-object semantics and demonstrations supply action support—but counterfactual trajectories can remain weakly separated or bend through the native-task direction before diverging. Existing remedies relabel data or steer internal activations, yet leave the instruction an implicit condition with no named motor meaning [5], [6], [7], [8], [9].

AMC (Fig. 1) grounds per-arm motion coordinates from atomic language and forward-kinematics (FK) supervision without vision. Each arm owns a spherical codebook of thirteen atoms: signed translations, signed rotations, and hold. Atomic text and normalized joint state condition the masked 50-step action flow and its low-frequency structure, tying atom identity to measured motion rather than object appearance. Under full observation, the coordinate specifies

a motion trend, while the action expert generates its scene-conditioned realization. A bounded tangent component adds contextual detail, and the ordered right/left pair conditions all 18 action-expert blocks through an independent, zero-initialized cross-attention path. The original self-attention and flow-time paths remain structurally intact, and the route is initially identity-preserving. The planner selects a subtask from a predefined vocabulary using a fixed prompt template; it outputs neither atoms nor weights. The low-level policy maps that subtask, images, and state to the motion coordinate.

Contact feedback adapts the ongoing motion without requiring a new semantic instruction. A shared encoder converts one second of wrench and state history into 30 slow tokens and the newest post-anchor samples into 10 fast tokens. The nominal z_M , augmented with the current real-time chunking (RTC) phase [10], queries this memory to produce the force correction z_F . A bounded spherical update regenerates only the unexecuted suffix of the 50-step horizon. Recomputing each update from the fixed z_M avoids accumulating corrections across contact updates.

Two boundaries scope our claim: we do not argue that a strong baseline ignores language, nor that an explicit coordinate can ground concepts absent from both pretraining and robot data. Throughout this paper, OOD denotes held-out recombinations of known targets, orders, and scene configurations. Within that scope, AMC raises opposite-atom separation from 39.1/24.0% to 92.5/83.1% for single/dual prompts over 7,520 per-policy matched offline interventions from recorded real-robot states, while retaining closed-loop competence that steering-oriented baselines substantially lose.

Our contributions are:

- **A geometry-supervised, intervenable language–motion interface.** FK-grounded per-arm coordinates connect atomic language to continuous action generation and expose a site for inspecting and intervening on motion under full observation.
- **A bounded update anchored in the same representation.** Force feedback revises the unexecuted action suffix through a bounded, non-accumulating spherical correction to the nominal motion coordinate.
- **Validation across three separated capability regimes.** Atomic interventions, fruit trials in familiar and held-out layouts, and contact-rich trials test geometric-direction steering, closed-loop subtask steering, and force adaptation.

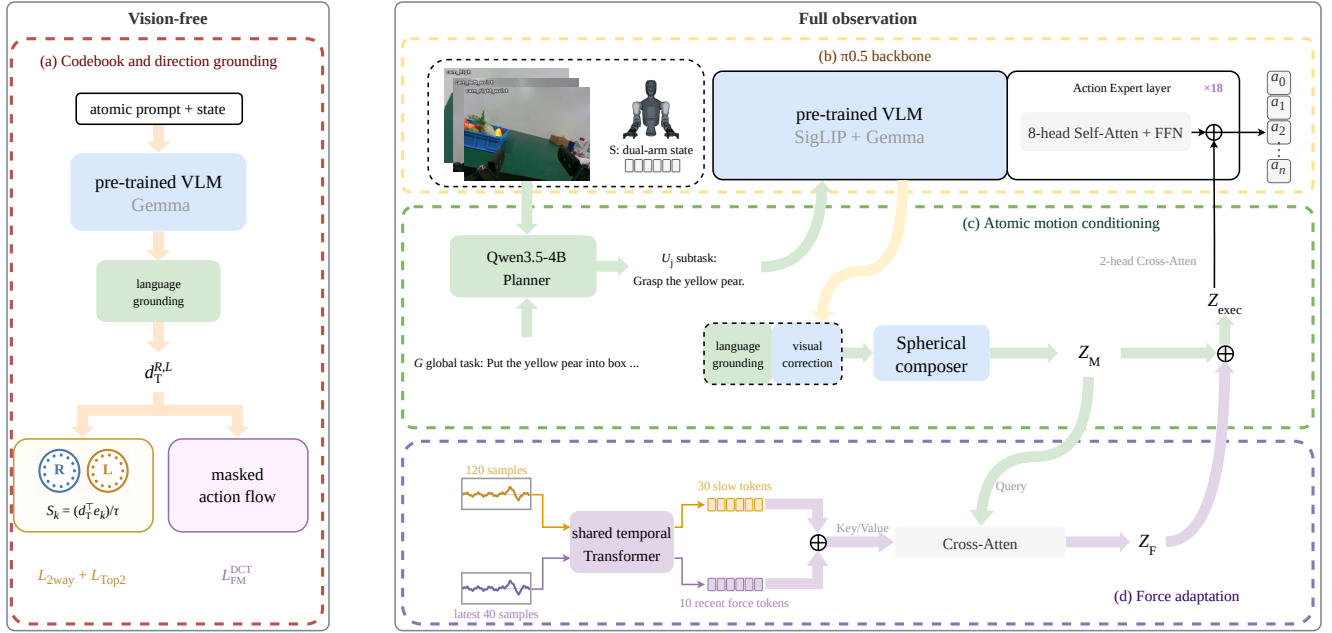


Fig. 1. Overview of AMC and its force-enabled extension. (a) Vision-free codebook and direction grounding. (b) $\pi_{0.5}$ backbone. (c) Atomic motion conditioning. (d) Force adaptation. Panels (b,c) support full-observation training and inference; the force-enabled variant adds (d). Losses in (b)–(d) are omitted for clarity.

II. RELATED WORK

A. Language-Conditioned VLA and Latent Actions

OpenVLA predicts discretized actions from a vision-language backbone, whereas π_0 and $\pi_{0.5}$ attach a continuous flow action expert to a pretrained vision-language model (VLM) [1], [2], [3], continuing the broader use of chunked, receding-horizon prediction in ACT and Diffusion Policy [11], [12]. Action abstractions have been learned by residual vector quantization, frequency-domain tokenization, and visual future prediction [13], [14], [15], and LARY argues that semantic decodability and physical reconstruction must be scored separately [16]. Our codebook differs in origin: it begins from an embodiment-specific geometric vocabulary rather than a learned token inventory, while the deployed policy still emits continuous trajectories.

Closest to us, LA4VLA removes vision during language-action pretraining, then returns to conventional full-observation VLA training [6]. It changes the curriculum but retains no fixed FK-derived dictionary, per-arm intervention variable, or contact-update site. We instead ground signed geometric axes, preserve their coordinates under full observation, and expose them directly to the action expert. Residual Semantic Steering (RSS) [17] combines Monte Carlo Syntactic Integration with Residual Affordance Steering to amplify language over a vision-only prior, but operates at the action-distribution level without a named per-arm coordinate. CAST and CofactVLA use counterfactual relabeling [5], [18]; other work steers internal activations or filters plans through latent-aligned VLM verification [7], [8]. None of these methods exposes a fixed, per-arm FK-grounded coordinate shared by language intervention and force adaptation.

B. Hierarchical Planning and Fast Contact Feedback

Hierarchical policies ground language into executable skills and revise plans across stages [19], [20]. We keep completed-state information in a compact recursive summary from Qwen3.5-4B [21], a boundary-rate planner rather than a controller: only its subtask reaches the motor policy.

On the contact side, RDP, ForceVLA, and FAVLA pair a slow visuomotor route with faster tactile or force conditioning [22], [23], [24]. AT-VLA and T-Rex route native-rate signals through specialized action paths [25], [26], while predictive models forecast tactile evolution before revising actions [27], [28]. FACTR shows that visually dominant policies can underutilize force without an explicit force-attending curriculum [29]. We instead place force correction in the same spherical space as language, recomputed from one fixed nominal latent.

III. METHOD

AMC comprises three functional components: a planner that converts a global task into one executable subtask at a time (Sec. III-A); a per-arm atomic coordinate that is first grounded from language and proprioception with vision withheld, then reused under full observation to condition every depth of the action expert (Sec. III-B–III-D); and a spherical force update that revises the coordinate during contact without displacing its semantic anchor (Sec. III-E). The low-level coordinate pathway is trained progressively through vision-free grounding, full-observation continuation, and force-adaptation stages.

A. Hierarchy and Action Interface

At each 1 Hz tick, the upper loop receives global task G , the current observation O_j , previous memory M_{j-1} , and

previous subtask U_{j-1} . Qwen3.5-4B emits

$$(M_j, U_j) \sim p_\psi(\cdot \mid G, O_j, M_{j-1}, U_{j-1}). \quad (1)$$

M_j is a compact cross-frame temporal anchor recording completed events and persistent object state. U_j is either the previous subtask carried forward or a new executable command when current visual evidence warrants a transition. Qwen3.5-4B is used without task-specific supervised fine-tuning (SFT); a fixed prompt template constrains its output to the predefined subtask vocabulary. Only U_j is passed to the low-level policy.

The low-level policy observes three RGB images I_t , the active subtask U_j , and dual-arm state

$$s_t = [q_t^L, g_t^L, q_t^R, g_t^R] \in \mathbb{R}^{16}. \quad (2)$$

It predicts $H = 50$ actions at 30 Hz through the 32-D $\pi_{0.5}$ interface; the first 16 coordinates control the two seven-DoF arms and grippers, while the remaining coordinates preserve base-checkpoint compatibility. The action expert retains the standard $\pi_{0.5}$ conditional flow-matching target [3].

One subtask typically spans several 50-step horizons; during training each arm-horizon receives either a strict single/dual atomic target or, when no strict atom is supported, a weighted Top-5 FK target used only during full-observation training. A subtask is thereby realized as a sequence of per-horizon motor targets (Sec. III-B). At inference, U_j , images, and state drive the low-level Q1/Q3 heads to predict codebook-aligned per-arm directions; the resulting inspectable motor coordinate can subsequently be revised by force (Sec. III-E).

B. An Atomic Vocabulary Defined by Forward Kinematics

The labeling rule must be immune to the language-motion correlation that we seek to break. We therefore derive every atomic target from measured geometry, never from subtask text. For each arm $a \in \{L, R\}$,

$$\mathcal{A} = \{\pm x, \pm y, \pm z, \pm r_x, \pm r_y, \pm r_z, \text{hold}\}. \quad (3)$$

The first six atoms denote signed translations and the next six signed rotations in the robot base frame. We compute end-effector motion with the deployment URDF and a 0.20-m tool-center-point (TCP) offset, using signed translation and the spatial rotation log of $R_h R_0^\top$ over each horizon. A horizon dominated by one component receives a single positive atom. When two compatible components jointly dominate, such as in a diagonal reach, it receives a weighted Top-2 target; forcing it onto one axis would assign a direction the arm did not take. Top-2 weighted labels are used in vision-free grounding. During full-observation training, an arm without a strict single- or dual-atom label instead uses a normalized Top-5 FK mixture target (Sec. III-D), without assigning an unsupported discrete atom. Because labels are derived from FK alone, object-aware phrasing cannot alter the supervision targets.

Each arm owns a normalized 512-D dictionary

$$E^a = \{e_1^a, \dots, e_{13}^a\}, \quad e_k^a \in \mathbb{S}^{511}. \quad (4)$$

Here, d_T^a denotes the vision-free, language-and-proprioception-grounded unit direction for arm $a \in \{R, L\}$. For positives P and all other atoms N , $\rho_k = (d_T^a)^\top e_k^a$, and $s_k = \rho_k / \tau_a$ is its temperature-scaled cosine similarity to atom k , with $\tau_a = 0.10$. The sample-wise multi-positive Two-Way loss is

$$\mathcal{L}_{2\text{way}}^a = \text{softplus}[\text{LSE}_{p \in P}(-s_p) + \text{LSE}_{n \in N}(s_n)]. \quad (5)$$

The two log-sum-exp terms push the weakest positive up and the strongest negative down within the same sample, so a dual-motion horizon can keep both of its atoms; a softmax objective over a single label would instead force the two to compete. For FK mixture weights $w_p \geq 0$ with $\sum_{p \in P} w_p = 1$, we define

$$u_P^a = \frac{\sum_{p \in P} w_p e_p^a}{\left\| \sum_{p \in P} w_p e_p^a \right\|}, \quad (6)$$

$$\mathcal{L}_{\text{Top2}}^a = 1 - (d_T^a)^\top u_P^a.$$

This weighted-cosine term aligns a single or dual direction with its normalized FK code mixture.

C. Vision-Free Atomic Grounding

This stage jointly grounds the atomic codebooks and motion directions using the $\pi_{0.5}$ VLA with atomic modules but no visual input (Fig. 1(a)). Withholding vision is the point of this phase, not a simplification. A policy that can see the scene may satisfy an atomic prompt by replaying whatever motion that scene usually elicits, leaving the atom no independent meaning. We therefore supply only the atomic prompt and the normalized robot state, encoded as the pretrained VLM state prefix. This removes the direct scene shortcut and forces the atomic instruction to disambiguate the desired motion direction given the current state. The Q1 and Q3 heads emit unit directions d_T^R, d_T^L that condition the same action expert used at deployment. We mask the horizon flow loss to the labeled arm's joints, leaving grippers, the opposite unlabeled arm, and padding coordinates untargeted, so a direction is never credited for motion it did not name.

To suppress 30-Hz action noise while preserving coarse motion, the flow matching target for the labeled arm is replaced by a discrete cosine transform (DCT-II) low-frequency reconstruction: $C^a = D_{0:K-1} Y^a$ with $K = 3$ (DC term and two lowest non-DC modes [14]) extracts three low-frequency coefficients; the length-50 reconstruction $\tilde{Y}^a = D_{0:K-1}^\top C^a$ then replaces the raw flow target. Gripper and unlabeled channels remain masked. This gives $\mathcal{L}_{\text{FM}}^{\text{DCT}}$, and the complete grounding objective is

$$\mathcal{L}_T = \mathcal{L}_{\text{FM}}^{\text{DCT}} + 0.10 \sum_a \mathcal{L}_{2\text{way}}^a + 0.10 \sum_a \mathcal{L}_{\text{Top2}}^a. \quad (7)$$

Thus atom identity is tied to both a 50-step physical action and its compact low-frequency structure, without image features explaining the label. At the end of this 25k-update grounding stage, the codebooks are frozen for full-observation continuation.

D. Full-Observation Motion Latent

Under full observation (Fig. 1(b,c)), the VLM receives the three camera views, robot state, and active subtask U_j . The atomic coordinate sets a motion trend, and the action expert supplies its scene-conditioned realization. Q1/Q3 predict right/left arm directions and Q2/Q4 their tangent-plane details. For arm a , the direction query yields $d_M^a \in \mathbb{S}^{511}$, while the detail query yields four bounded coefficients $c^a = \tanh(W_c h_{\text{detail}}^a)$. We project four learned seeds into the tangent plane of d_M^a and orthonormalize them into the four-vector basis $B_{\perp}^a = \{B_{\perp,k}^a\}_{k=1}^4$. Let $r_M^a = 0.5 \sum_{k=1}^4 c_k^a B_{\perp,k}^a$. Writing $r_{\perp} = (I - bb^{\top})r$ for the tangent projection, the bounded spherical update is

$$\theta = \theta_{\max} \tanh\left(\frac{\|r_{\perp}\|}{\theta_{\max}}\right) \\ \mathcal{S}(b, r; \theta_{\max}) = \text{Norm}\left(\cos \theta b + \sin \theta \frac{r_{\perp}}{\|r_{\perp}\|}\right). \quad (8)$$

Here $b \in \mathbb{S}^{511}$, Norm is ℓ_2 normalization, and the zero-residual case is defined by continuity as $\mathcal{S}(b, 0; \theta_{\max}) = b$ (the implementation uses a numerical stabilizer). Although r_M^a is tangent by construction, the generic projection is retained because the same update is reused for the force residual in Sec. III-E. The full-observation coordinate is $z_M^a = \mathcal{S}(d_M^a, r_M^a; \pi/4) \in \mathbb{S}^{511}$. Strict single/dual horizons use stopped-gradient d_T^a as teacher. A non-strict horizon instead uses the frozen FK mixture $u_{T_5}^a = \text{Norm}(\sum_{p \in T_5^a} w_p c_p^a)$ from its five largest normalized components, with loss $1 - (d_M^a)^{\top} u_{T_5}^a$; no unsupported atom is assigned. A 0.005-weighted 20° free cone penalizes geodesic excess between d_M^a and z_M^a . The 45° cap bounds detail around d_M^a , while teacher alignment anchors it to the grounded target. Thus full-observation contextual detail can shape the trajectory without changing latent norm. The ordered pair $[z_M^R, z_M^L]$ conditions every action-expert block through a separate two-head cross-attention residual:

$$H_{\ell} \leftarrow H_{\ell} + \text{CA}_{\ell}(H_{\ell}, [z_M^R, z_M^L]), \quad \ell = 1, \dots, 18. \quad (9)$$

Its output projection is zero-initialized. The backbone’s eight-head self-attention, feed-forward blocks, and flow-time AdaRMS remain unchanged.

During final stabilization, the direction supplied to the action expert is replaced by its stopped-gradient teacher (d_T for atomic labels and the frozen Top-5 mixture otherwise), while d_M remains cosine-supervised to that teacher. This prevents early visual drift from destabilizing action learning; the tangent detail remains predicted from full observation. At inference, the learned d_M is used without teacher substitution.

The flow objective alone does not guarantee strong utilization of the coordinate: the pretrained action expert can initially reduce the loss while largely bypassing the zero-initialized cross-attention path. We therefore add a matched counterfactual pass: a second run replaces one supervised arm direction with a hard-negative codebook atom outside

that arm’s positive support, holding every other input and the flow time fixed. With per-arm flow losses ℓ_{corr} and ℓ_{wrong} ,

$$\mathcal{L}_{\text{cf}} = [m + \ell_{\text{corr}} - \ell_{\text{wrong}}]_+, \quad m = 2 \times 10^{-3}. \quad (10)$$

The wrong direction must therefore reconstruct the demonstration strictly worse than the correct one by a margin m ; once that gap holds, the hinge is inactive and the term stops competing with the flow objective. The final continuation weights it at 0.10, alongside the full flow loss, cosine alignment between the predicted direction and its stopped-gradient teacher, and Top-5 weighted-cosine supervision.

E. Force-Responsive Latent Modulation

The force-enabled variant augments this full-observation policy with the force-adaptation branch (Fig. 1(d)). The force branch predicts a residual z_F that adapts nominal z_M to contact without requiring a new subtask instruction. For a 50-step rollout, RTC offset $i \in \{0, 10, 20, 30, 40\}$ denotes the number of already committed actions at a decision boundary. Each wrist provides a calibrated six-axis wrench at 120 Hz. Per arm, non-overlapping groups of four consecutive force-state samples are projected into 512-D tokens (120 Hz $\rightarrow$ 30 Hz), then passed through a shared two-layer temporal transformer and an arm-specific projection. The newest 120 raw samples yield 30 slow tokens $F_{\text{slow}}^a \in \mathbb{R}^{30 \times 512}$. At offset $i > 0$, the most recent at most 40 force-state samples acquired after the preceding decision anchor form 10 fast tokens $F_{\text{fast}}^{a,(i)} \in \mathbb{R}^{10 \times 512}$; unavailable fast positions at $i = 0$ are validity-masked. The paths share encoder weights and concatenate only in cross-attention. Before learning the action correction, B1 trains only this encoder with an auxiliary single-layer GRU decoder: the temporal transformer compresses slow force-state history into per-arm tokens, and the GRU decodes them into future force increments, shaping the encoder so slow tokens carry contact-predictive structure. The GRU is discarded at deployment. B2 freezes the VLM, nominal motion queries, codebooks, atomic cross-attention, and B1 force encoder; the future predictor is excluded from deployment. It then trains only the action expert and force adapter under RTC. z_M^a plus a sinusoidal RTC-phase embedding ϕ_i forms a query over the 30+10 token memory. A two-head cross-attention adapter and zero-initialized output produce one direct 512-D residual:

$$q^{a,(i)} = \text{LN}(W_q z_M^a + \phi_i), \quad (11)$$

$$z_F^{a,(i)} = W_o \text{swish}\left(W_h \text{CA}_2(q^{a,(i)}, [F_{\text{slow}}^a; F_{\text{fast}}^{a,(i)}])\right), \quad (12)$$

$$z_{\text{exec}}^{a,(i)} = \mathcal{S}(z_M^a, z_F^{a,(i)}; \pi/4). \quad (13)$$

The update uses the same 20° free cone, 45° cap, and 0.005 hinge weight as the visual route. Each $z_F^{a,(i)}$ is recomputed from nominal z_M^a , not added to a previous z_{exec} : this prevents recursive drift from the named atom and bounds total departure to one residual. The ordered $[z_{\text{exec}}^R, z_{\text{exec}}^L]$ pair replaces the nominal pair in Eq. (9) at every action block.

At training offset i , the demonstrated prefix of length i is clamped and excluded from loss; only the regenerated

TABLE I
TRAINING SCHEDULE. STEPS DENOTE PER-STAGE UPDATES.

Stage	Trainable	Steps
z_T grounding	VLA + atomic modules	25k
z_M full observation	VLA + atomic modules	25k / 20k*
B1 prediction (force only)	force encoder + predictor	5k
B2 adaptation (force only)	action expert + force adapter	5k

VLA: backbone + action expert. Atomic modules: queries, heads, codebooks, and z_M cross-attention. Codebooks freeze after z_T . *20k z_M for the force-enabled recipe.

suffix $i:H$ is supervised with uniform per-step weighting. At deployment, each boundary retains the committed prefix and regenerates only the unexecuted suffix [10]; force is sampled at 120 Hz, but policy updates occur once per ten executed actions (≈ 3 Hz).

F. Training and Model Size

Both recipes begin with 25k vision-free z_T updates, after which codebooks are frozen (Table I). The force-free recipe uses 25k z_M updates; the force-enabled recipe uses 20k z_M updates followed by B1 and B2. B1 trains only the force encoder and future-wrench predictor for 5k updates. B2 trains the action expert and force adapter for 5k RTC updates with the encoder frozen. Both recipes thus have 25k post-grounding action-expert updates; B1 adds encoder-only pretraining, not action-expert updates.

All stages use 8×H800 GPUs, global batch size 256, and cosine annealing. The z_T/z_M stages use peak lr 2.5×10^{-5} (minimum 2.5×10^{-6} ; 1000 warmup steps); B1 uses peak lr 10^{-4} (minimum 10^{-5} ; 300 warmup steps). B2 uses 5×10^{-6} to 10^{-6} with 200 warmup steps. The gap in Table III is not a causal force estimate: Table IV isolates the route by varying only force content within one checkpoint.

The deployed atomic and force routes add 34.56M and 12.40M parameters, totaling 46.96M (1.40% over the 3.353B $\pi_{0.5}$ backbone); the DCT-based target construction and B1 future-wrench predictor are training-only.

IV. EXPERIMENTS

A. Platform, Data, and Four Task Families

Experiments use a CR1 dual-arm platform with two seven-DoF arms, two grippers, a base camera, two wrist cameras, and a six-axis force/torque sensor at each wrist. RGB, state, and actions are recorded at 30 Hz; wrench is recorded at 120 Hz. FK annotation, evaluation, and visualization use the same URDF and 0.20-m TCP. The unified training corpus contains 2,375 episodes and 3.07M valid 30-Hz training samples across cabinet, drawer, fruit, mixed manipulation, screwdriver, plug, and vase domains. Normalization statistics use the training split only. On a single RTX 4090, atomic-only inference takes 117.4 ms; force-enabled slow/fast loops take 118.7/45.5 ms, within the 333 ms ten-action window.

We evaluate four families. **Multi-fruit sorting** evaluates closed-loop subtask steering: at annotated semantic boundaries, an operator selects an object- or destination-specific

instruction from the same predefined execution-subtask vocabulary, and the policy is evaluated on whether it follows that instruction. Fruit positions are randomized across familiar configurations. **Drawer retrieval and plug insertion** (hereafter **Plug**) requires opening the drawer, grasping and reorienting the power adapter, aligning the connector, and completing a contact-rich insertion, with execution subtasks supplied by the Qwen3.5-4B planner. **OOD fruit ordering** tests the same subtask steering under unseen fruit placement and target-box positions. **Bimanual vase wiping** alternates stabilization, rotation, and sustained wiping contact, with subtasks supplied by the Qwen3.5-4B planner.

B. Baselines and Metrics

We compare six policies: fine-tuned $\pi_{0.5}$ [3], matched reproductions of LA4VLA [6] and ForceVLA [23], an adaptation of RSS [17] to our real-robot setting, our force-free AMC, and AMC + Force. All reproductions use the same three-view RGB observations, 16-D dual-arm state, 32-D action interface, and evaluation protocol; training subsets are matched to each method’s evaluated task families. For real-robot task progress, LA4VLA and RSS are evaluated only on fruit tasks; ForceVLA only on Plug and Vase.

The primary robot metric is mean normalized task progress. Each method is evaluated in 50 trials per task. If trial i earns s_i out of the task’s prespecified maximum S , its progress is $100s_i/S$. Plug progress uses four one-point macro-stages: opening the drawer, establishing the grasp on the adapter, aligning the connector, and completing insertion. The planner may issue multiple finer-grained execution subtasks within each scored stage. Vase uses five points: two for rotation (one per half-turn, stabilization subsumed), two for wiping (one per half-clean), and one for returning the vase. Scoring and termination conditions are fixed across methods. For fruit tasks, an operator selects the execution subtask from the same predefined vocabulary at annotated semantic boundaries; task progress measures whether the policy follows it. For Plug and Vase, Qwen3.5-4B generates subtask prompts automatically. Fruit and contact-task results measure distinct capabilities and are not pooled into a single average. Atomic steerability follows the matched protocol of Sec. IV-C. Planner reliability is evaluated separately at subtask boundaries.

C. Atomic Steering

Figure 2 makes the counterfactual visible: with images and state fixed, native trajectories expose the scene-associated task prior, Empty removes text, and changing only the signed atom separates endpoints or rotations. Table II quantifies this contrast. We cluster standardized 14-D joint states (grippers excluded) into 250 clusters. Retaining outcome-agnostic modes observed at least five times yields 1,880 cluster-mode pairs (830 single, 1,050 dual) across 948 states (452 left, 496 right). Four states per pair yield 7,520 interventions per policy (3,320 single, 4,200 dual). All policies use the same interventions; within a policy, images, state, normalization, sampler, and initial flow noise are fixed as prompts change.

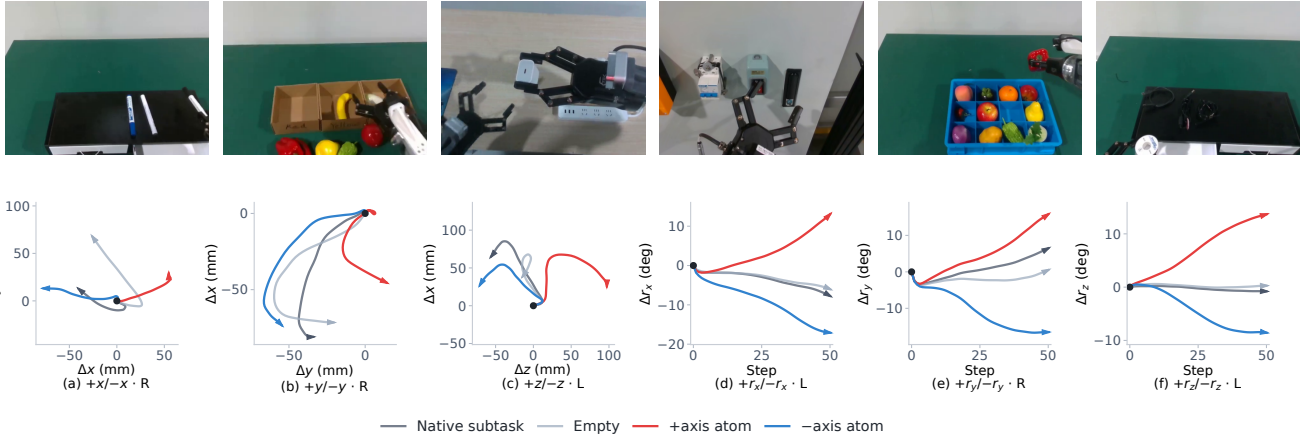


Fig. 2. Single-atom prompt interventions with fixed images and state. Native, Empty, and opposite-signed prompts vary one atom. Panels (a)–(c) show horizon-50 base-frame TCP paths; (d)–(f) show signed rotation trajectories. Dots and arrowheads mark common starts and endpoints. Panels (a)–(f) use marker insertion, apple grasp, power-strip hold, switch press, chili placement, and solder-coil placement states, respectively.

TABLE II

MATCHED HORIZON-50 ATOMIC STEERING AT $10 \text{ mm}/2^\circ$ (%). AMC USES THE PRECEDING NON-SPHERICAL 50K CHECKPOINT ($25\text{k } z_T + 25\text{k } z_M$). THE SOURCE-SUPPORT FILTER IS APPLIED TO EACH POLICY.

Method	Atom type	Absolute	Vs. Empty	Pair sep.
Fine-tuned $\pi_{0.5}$	Single	50.5	18.3	18.1
	Dual	37.1	6.9	8.1
LA4VLA-style	Single	52.9	31.7	39.1
	Dual	37.9	13.2	24.0
RSS	Single	52.6	39.2	60.7
	Dual	26.2	9.7	22.5
AMC	Single	80.2	74.5	92.5
	Dual	57.9	39.1	83.1

Empty uses the same images and state with no text; FK maps predictions to TCP without execution.

A translation or rotation intervention succeeds when its horizon-50 endpoint moves in the requested signed base-frame direction beyond threshold; a dual combines two compatible components on one target arm (the other holds) and requires both. Pair separation instead requires correctly ordered, threshold-separated signed-atom endpoints, without either endpoint individually clearing threshold (Table II). Vs. Empty applies the signed threshold to the prompted endpoint relative to Empty. Pair separation uses the reversal-supported subset: 2,316 single and 2,224 dual interventions. Absolute tests requested motion, Vs. Empty isolates language beyond the same visual prior, and pair separation tests opposite-prompt ordering.

The $10 \text{ mm}/2^\circ$ threshold acts only as a noise gate: retained interventions have mean controlled-axis magnitudes of 55.5 mm and 20.2° .

Fine-tuned $\pi_{0.5}$ reaches 50.5/37.1% Absolute success but only 18.3/6.9% Vs. Empty and 18.1/8.1% pair separation (single/dual): native motion direction alone does not establish language control. The gap shows that a correct absolute motion can still follow the state- and scene-conditioned prior rather than the prompted atom. LA4VLA-style adds

21.0/15.9 points of pair separation over $\pi_{0.5}$, but only 2.4/0.8 points of Absolute success and does not preserve closed-loop fruit progress (Table III). RSS has the strongest baseline single-atom pair separation (60.7%), yet its dual Absolute/Vs. Empty/pair scores fall to 26.2/9.7/22.5%; this does not isolate residual interference as the cause. AMC leads all metrics: 80.2/57.9% Absolute, 74.5/39.1% Vs. Empty, and 92.5/83.1% pair separation. The Single–Dual gap reflects the joint criterion and possibly different state/mode difficulty. Thus the complete coordinate pathway improves prompt sensitivity and directional correctness, including compatible two-component requests. These gains reflect the complete coordinate pathway; the following representation analysis checks that its intended code geometry is retained rather than assigning the effect to one module.

D. Latent-Space Analysis

Figure 3 visualizes 900 single-atom and 900 compatible-dual d_M^a directions per arm. Before t-SNE, in the original 512-D space, the labeled atom is nearest for every single sample, and the target pair is the two nearest codes for 99.1%/99.7% of right/left dual samples. Mean angular errors are $4.96^\circ/6.37^\circ$ for single atoms and $14.91^\circ/11.84^\circ$ to the normalized weighted Top-2 target for duals. Mean codebook separation is $94.61^\circ/94.66^\circ$, indicating no collapse. The t-SNE panels are qualitative; nearest-code and angle measures are computed before projection.

With observation, VLM prefix cache, diffusion state, and noisy action fixed, replacing spherical z_M by its nominal direction changes the final action-expert hidden state by 8.9/10.1/10.3% RMS on Fruit/Plug/Vase; batch-shuffling z_M changes it by 12.7/12.8/14.0% (32 native-subtask samples per domain). Thus the spherical coordinate materially modulates this representation.

The single-atom sweep contains 1,100 left-arm and 2,220 right-arm interventions per policy. Anchor selection is balanced, but unequal source support means the pooled rate is not an equal-weight arm average.

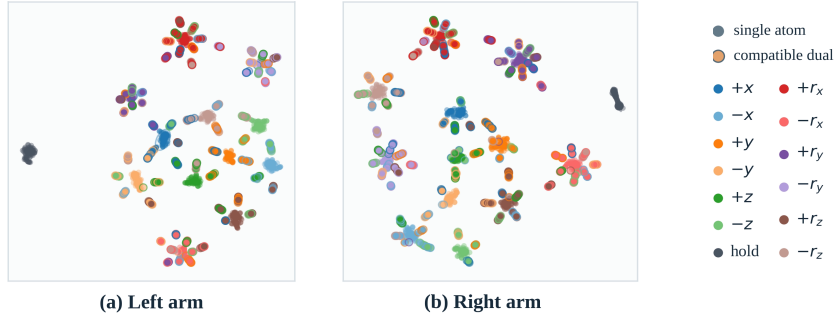


Fig. 3. Per-arm t-SNE of grounded directions d_M^a . Colors mark target atoms; circles and two-tone circles denote single and dual samples.

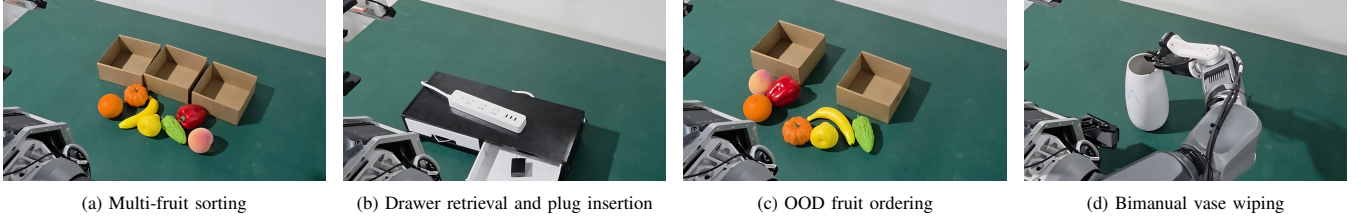


Fig. 4. Real-robot setups for the four experiment families. The two fruit scenes differ: (a) measures closed-loop subtask steering under human-provided prompts with randomized familiar positions; (c) uses the same protocol with unseen fruit placement and target-box positions.

TABLE III

MEAN NORMALIZED TASK PROGRESS (%) OVER 50 TRIALS ON THE FOUR REAL-ROBOT SETUPS IN FIG. 4; “–” DENOTES UNTESTED PAIRS.

Method	Fruit / multi-sort	Fruit target/order (OOD)	Plug (4-point)	Vase (5-point)
Fine-tuned $\pi_{0.5}$	82.7	60.5	55.0	65.2
RSS	54.5	39.1	–	–
LA4VLA-style	60.7	40.4	–	–
ForceVLA	–	–	30.0	40.4
AMC	88.9	87.8	59.0	71.6
AMC + Force	–	–	78.5	75.2

E. Closed-Loop Task Progress

Figure 4 shows the four closed-loop scenes and Table III reports their task progress. The fruit families test whether object- and destination-specific subtasks can use per-arm motion trends under visual context. Unlike Table II, these trials prescribe neither an atomic axis nor an endpoint: the policy must identify the named object or destination from images and compose the corresponding trends across successive horizons. Thus fixed-state geometric redirection in Table II is tested here as object- and destination-level subtask composition into scene-conditioned task progress. Fine-tuned $\pi_{0.5}$ retains a competent visual motor prior (82.7/60.5% familiar/OOD), but its lower prompt-contrast scores indicate limited redirection of that prior when target positions and boxes change. AMC preserves strong visual performance while exposing an FK-grounded per-arm coordinate; progress rises to 88.9/87.8%.

The two fruit baselines fail differently. LA4VLA-style improves offline prompt contrast, but its language–direction binding is not tied to a fixed geometric coordinate after full-observation training; its limited Absolute gains and 60.7/40.4% execution are consistent with visual-control interference. RSS has the strongest baseline single-atom separation, but one action-distribution residual rather than independent bilateral coordinates; its weak dual steering and 54.5/39.1% fruit progress illustrate that separable prompt

responses alone do not guarantee executable subtask composition.

Relative to fine-tuned $\pi_{0.5}$ (55.0/65.2%), force-free AMC already reaches 59.0/71.6% on Plug/Vase, while AMC + Force with matched history raises these to 78.5/75.2%. Plug gains more (19.5 points), consistent with discrete drawer, alignment, and insertion contacts benefiting from RTC suffix correction. On Vase, force-free AMC already reaches 71.6%, indicating that visual context and the per-arm action composition supplied by AMC account for much of task execution; matched force history then yields a smaller, but positive, 3.6-point incremental gain. ForceVLA reaches 30.0/40.4% in our matched reproduction; this result is consistent with, but does not establish, a mismatch between direct force routing and the pretrained visuomotor prior. AMC instead makes a bounded, non-accumulating update to its motion coordinate and regenerates only the suffix; Table IV distinguishes current contact from phase.

With non-force inputs fixed, shifted history preserves RTC phase but misaligns current contact; its 0.0581/21.01 to 0.0459/19.41 rad/mm improvement captures generic phase-conditioned correction. Matched history reaches 0.0349/15.27 (23.9/21.4% relative to shifted), isolating current force content.

Planner reliability is evaluated separately at subtask boundaries: without task-specific SFT, Qwen3.5-4B produces

the correct next subtask within five successive 1 Hz queries in 94.6%/92.4% of 500 Vase/Plug cases. This boundary-level score does not measure end-to-end task success.

TABLE IV
FORCE ABLATION ON 50 PLUG/VASE TRIALS (SUFFIX-WEIGHTED
RMSE; LOWER IS BETTER).

Force condition	Joint (rad)	TCP (mm)
Route disabled	0.0581	21.01
Half-episode shifted history	0.0459	19.41
Matched force history	0.0349	15.27

V. LIMITATIONS AND CONCLUSION

The thirteen atoms specify dominant base-frame trends, not complete trajectories: grasp configuration and fine motion remain with the action expert. Generalization depends on semantic and action support in pretraining and demonstrations, and FK-derived labels inherit calibration error. Force adapts generated motion but does not explicitly regulate wrench or guarantee contact stability; planner reliability is boundary-level rather than end to end.

Within these limits, AMC organizes existing semantic and action support into named per-arm motion coordinates that condition every action-expert block. A bounded, non-accumulating force update revises these coordinates during contact while retaining the nominal semantic anchor. Our evaluations support improved single- and compatible dual-direction steering, closed-loop fruit subtask steering, and contact-rich task progress, providing an interpretable interface for language guidance and force-conditioned motion adaptation.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2679–2713.
- [2] K. Black, N. Brown, D. Driess, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [3] K. Black, N. Brown, J. Darpinian, *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 17–40.
- [4] Z. Li, L. Ren, J. Yang, *et al.*, “VIP: Vision instructed pre-training for robotic manipulation,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 35 769–35 778.
- [5] C. Glossop, W. Chen, A. Bhorkar, D. Shah, and S. Levine, “CAST: Counterfactual labels improve instruction following in vision-language-action models,” *arXiv preprint arXiv:2508.13446*, 2025.
- [6] T. Lin, Y. Du, Y. Mao, *et al.*, “LA4VLA: Learning to act without seeing via language-action pretraining,” *arXiv preprint arXiv:2606.27295*, 2026.
- [7] B. Häon, K. C. Stocking, I. Chuang, and C. Tomlin, “Mechanistic interpretability for steering vision-language-action models,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 2743–2762.
- [8] Y. Wu, T. Tian, G. Swamy, and A. Bajcsy, “From foresight to forethought: VLM-in-the-loop policy steering via latent alignment,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [9] Y. Fang *et al.*, “When vision overrides language: Evaluating and mitigating counterfactual failures in VLAs,” *arXiv preprint arXiv:2602.17659*, 2026.
- [10] K. Black, A. Z. Ren, M. Equi, and S. Levine, “Training-time action conditioning for efficient real-time chunking,” *arXiv preprint arXiv:2512.05964*, 2025.
- [11] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [12] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [13] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, “Behavior generation with latent actions,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 26 991–27 008.
- [14] K. Pertsch, K. Stachowicz, B. Ichter, *et al.*, “FAST: Efficient action tokenization for vision-language-action models,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [15] S. Ye, J. Jang, B. Jeon, *et al.*, “Latent action pretraining from videos,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [16] D. Nie, F. Chen, Q. Lv, J. Kuang, X. Li, X. Cao, and X. Cai, “LARY: A latent action representation yielding benchmark for generalizable vision-to-action alignment,” *arXiv preprint arXiv:2604.11689*, 2026.
- [17] Z. Zhan, Y. Chen, J. Zhou, Q. Lyu, H. Liu, K. Wang, L. Lin, and G. Wang, “Stable language guidance for vision–language–action models,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, San Diego, CA, USA, 2026, pp. 4137–4159.
- [18] Y. Zhang, Y. Wu, H. Duan, and J. Han, “CofactVLA: Deconfounding vision-language-action models via counterfactual intervention,” *arXiv preprint arXiv:2608.04396*, 2026.
- [19] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318.
- [20] Y. Feng, J. Han, Z. Yang, X. Yue, S. Levine, and J. Luo, “Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 2038–2062.
- [21] Qwen Team, “Qwen3.5-4B,” Hugging Face model card, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.5-4B>
- [22] H. Xue, J. Ren, W. Chen, *et al.*, “Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [23] J. Yu, H. Liu, Q. Yu, *et al.*, “ForceVLA: Enhancing VLA models with a force-aware MoE for contact-rich manipulation,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [24] Y. Li, P. Tang, W. Zhang, *et al.*, “FAVLA: A force-adaptive fast-slow VLA model for contact-rich robotic manipulation,” *arXiv preprint arXiv:2602.23648*, 2026.
- [25] X. Li, M. Cai, J. Xu, J. Zhu, H. Fan, *et al.*, “AT-VLA: Adaptive tactile injection for enhanced feedback reaction in vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 28 764–28 774.
- [26] D. Niu, Z. Liu, Z. Wang, B. Shao, Z.-H. Yin, A. Pai, Y. Sharma, S. Saravalle, *et al.*, “T-Rex: Tactile-reactive dexterous manipulation,” *arXiv preprint arXiv:2606.17055*, 2026.
- [27] J. Zhou, F. Hong, Y. Li, *et al.*, “TouchWorld: A predictive and reactive tactile foundation model for dexterous manipulation,” *arXiv preprint arXiv:2607.07287*, 2026.
- [28] S. Zhang, X. Zhang, Y. Shen, Y. Li, Y. Gao, S. Zhang, Y. Zhang, K. Long, J. Wu, J. Pan, J. Deng, and Y. Zhang, “ReTouch: Empowering contact-rich dexterous manipulation with online-refined tactile prediction,” *arXiv preprint arXiv:2608.01824*, 2026.
- [29] J. J. Liu, Y. Li, K. Shaw, T. Tao, R. Salakhutdinov, and D. Pathak, “FACTR: Force-attending curriculum training for contact-rich policy learning,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.